

PROPUESTA DE NUEVA ESTRUCTURA

Matriz dispersa con listas ortogonales

Análisis funcional y prototipo para incorporarla a DSA Lab como una visualización educativa, interactiva y sincronizada con código Java.

Decisión recomendada. Usar cabeceras circulares para filas y columnas. Cada celda distinta de cero se representa una sola vez y posee dos enlaces: right para la fila y down para la columna.

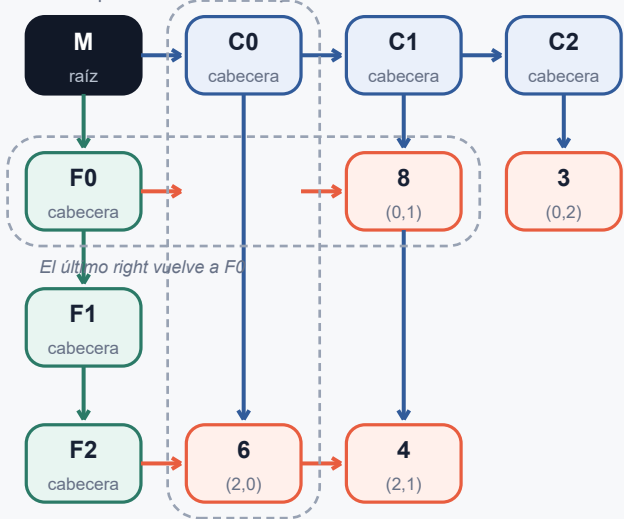
Matriz lógica 3 x 3

Sólo almacena 4 nodos.

0	8	3
0	0	0
6	4	0

Representación ortogonal

Una celda pertenece a una fila y una columna.



Objetivo

Entender cómo una misma celda participa en dos listas.

Público

Alumnos que recién estudian nodos, referencias y matrices.

Alcance

Insertar, actualizar, eliminar, buscar y recorrer.

1. Qué estructura se va a enseñar

Una matriz dispersa guarda principalmente ceros. En una matriz tradicional se reserva memoria para cada posición, incluso cuando su valor es cero. En la representación ortogonal sólo existen nodos para las celdas con un valor distinto de cero.

Modelo de nodos

```
class Nodo {
    int fila;
    int columna;
    int valor;
    Nodo right; // siguiente valor de la misma fila
    Nodo down;  // siguiente valor de la misma columna
}
```

- right mantiene cada fila ordenada de izquierda a derecha por número de columna.
- down mantiene cada columna ordenada de arriba hacia abajo por número de fila.
- Cada fila y columna tiene una cabecera. La lista termina cuando vuelve a esa cabecera.
- Una celda se comparte: no se duplica para la fila y para la columna.

Aclaración importante sobre up/down

Si la estructura debe seguir siendo simple circular, no necesita enlaces left ni up. Para regresar se sigue avanzando hasta volver a la cabecera. Agregar up convertiría la dimensión vertical en una lista doble. Por eso el modelo recomendado usa right y down. En la interfaz se pueden rotular como “siguiente en fila” y “siguiente en columna”.

Invariantes que la animación debe respetar

Orden

right ordena columnas y down ordena filas.

Circularidad

Ninguna lista termina en null.

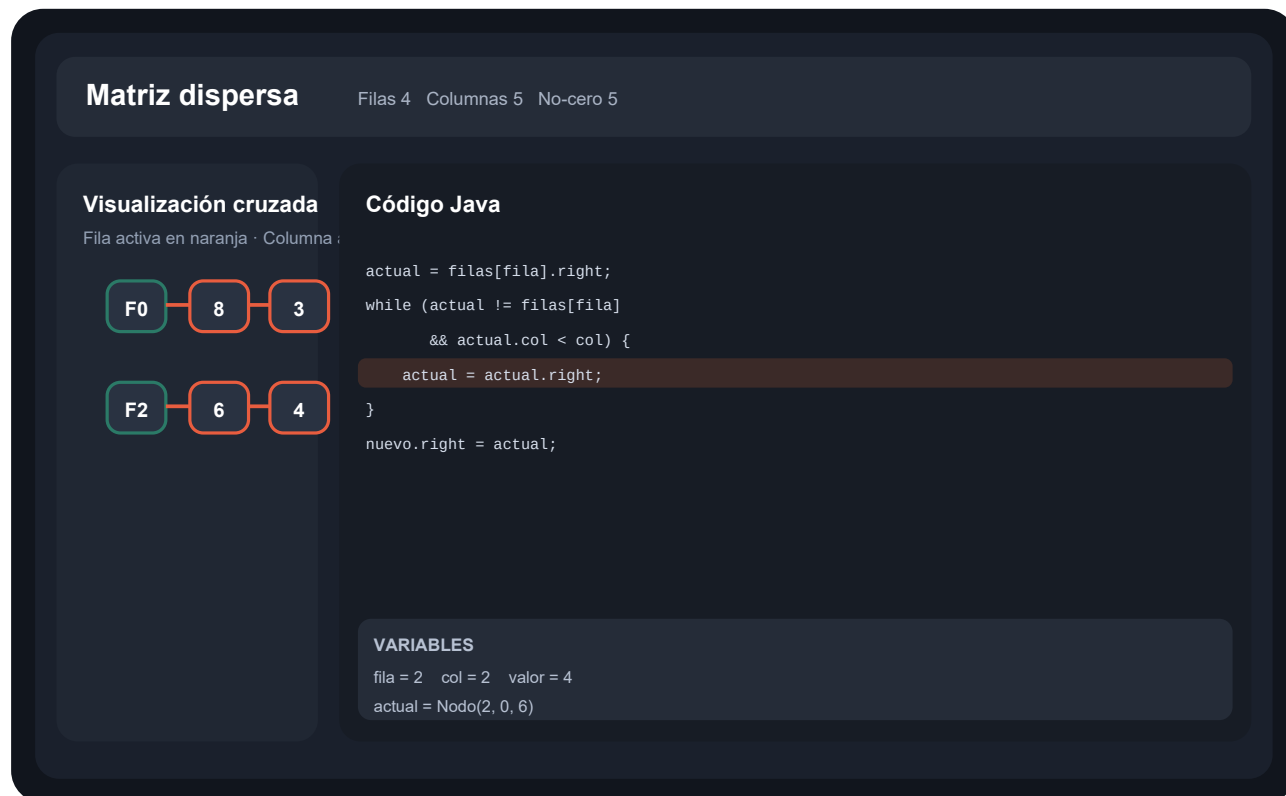
Unicidad

No hay dos nodos para la misma coordenada.

Regla de valor cero: insertar 0 equivale a eliminar la celda. Así la estructura conserva únicamente los datos que aportan información.

2. Prototipo dentro de DSA Lab

La nueva sección puede ubicarse en Estructuras especializadas con el nombre “Matriz dispersa”. El centro de la experiencia no será una cuadrícula llena, sino el cruce entre la fila y la columna que se está recorriendo.



Controles recomendados

Campo	Uso	Validación visible
Filas / columnas	Crear o redimensionar la matriz	Entre 2 y 8 para conservar legibilidad
Fila, columna, valor	Insertar o actualizar una celda	Índices desde 0; valor entero
Buscar	Localizar una coordenada	Muestra 0 si no existe nodo
Eliminar	Desconectar de fila y columna	Debe actualizar ambos enlaces
Recorrer fila / columna	Seguir right o down	Se detiene al volver a la cabecera

Animación de una inserción

1. Resaltar la cabecera de la fila y avanzar con right.
2. Resaltar la cabecera de la columna y avanzar con down.
3. Crear el nodo en el punto de cruce, todavía sin conectarlo.
4. Conectar primero los dos enlaces de fila y después los dos enlaces de columna.
5. Confirmar el nuevo valor en la cuadrícula lógica y en el panel de variables.

3. Operaciones y complejidad

Sea F la cantidad de nodos no-cero en la fila y C la cantidad de nodos no-cero en la columna. La implementación recorre listas cortas, no todas las posiciones de la matriz.

Operación	Qué se anima	Costo
insertar(f, c, v)	Busca predecesor en fila y columna; enlaza el nodo	$O(F + C)$
obtener(f, c)	Avanza por right hasta igualar o superar la columna	$O(F)$
eliminar(f, c)	Desconecta el mismo nodo de ambas listas	$O(F + C)$
recorrerFila(f)	Sigue right hasta volver a la cabecera	$O(F)$
recorrerColumna(c)	Sigue down hasta volver a la cabecera	$O(C)$
mostrarDensa()	Consulta todas las coordenadas	$O(\text{filas} \cdot \text{columnas})$
memoria	Cabeceras más nodos no-cero	$O(\text{filas} + \text{columnas} + \text{nz})$

Estados que deben aparecer en el panel de variables

Entrada fila, columna y valor.	Fila anteriorFila y actualFila.	Columna anteriorCol y actualCol.
--	---	--

- nz : cantidad total de nodos con valores distintos de cero.
- fase : BUSCAR_FILA, BUSCAR_COLUMNA, ENLAZAR_FILA, ENLAZAR_COLUMNA o LISTO.
- línea activa: debe corresponder a la referencia que acaba de cambiar en la animación.

Casos límite que deben enseñarse

- Primera celda de una fila o columna vacía.
- Inserción antes del primer nodo, entre dos nodos y al final circular.
- Actualizar una coordenada que ya existe, sin crear otro nodo.
- Eliminar el único nodo de una fila o columna.
- Ingresar cero para retirar una celda existente.
- Rechazar índices negativos o fuera del tamaño de la matriz.

4. Código Java educativo - estructura base

El código se mantiene deliberadamente explícito. No usa colecciones avanzadas; cada cambio de referencia puede iluminarse y verse en la estructura.

```
public class MatrizDispersa {
    static class Nodo {
        int fila;
        int columna;
        int valor;
        Nodo right;
        Nodo down;

        Nodo(int fila, int columna, int valor) {
            this.fila = fila;
            this.columna = columna;
            this.valor = valor;
        }
    }

    private final int cantidadFilas;
    private final int cantidadColumnas;
    private final Nodo[] filas;
    private final Nodo[] columnas;
    private int cantidadNoCeros;

    public MatrizDispersa(int cantidadFilas, int cantidadColumnas) {
        if (cantidadFilas <= 0 || cantidadColumnas <= 0) {
            throw new IllegalArgumentException("El tamaño debe ser positivo");
        }

        this.cantidadFilas = cantidadFilas;
        this.cantidadColumnas = cantidadColumnas;
        this.filas = new Nodo[cantidadFilas];
        this.columnas = new Nodo[cantidadColumnas];

        for (int fila = 0; fila < cantidadFilas; fila++) {
            Nodo cabecera = new Nodo(fila, -1, 0);
            cabecera.right = cabecera;
            filas[fila] = cabecera;
        }

        for (int columna = 0; columna < cantidadColumnas; columna++) {
            Nodo cabecera = new Nodo(-1, columna, 0);
            cabecera.down = cabecera;
            columnas[columna] = cabecera;
        }
    }
}
```

Lectura visual: una cabecera de fila se apunta a sí misma con right cuando está vacía; una cabecera de columna se apunta a sí misma con down. Por eso nunca se compara con null para saber si terminó el recorrido.

5. Código Java educativo - insertar y actualizar

```
public void insertar(int fila, int columna, int valor) {
    validarPosicion(fila, columna);

    if (valor == 0) {
        eliminar(fila, columna);
        return;
    }

    Nodo cabeceraFila = filas[fila];
    Nodo anteriorFila = cabeceraFila;
    Nodo actualFila = cabeceraFila.right;

    while (actualFila != cabeceraFila
        && actualFila.columna < columna) {
        anteriorFila = actualFila;
        actualFila = actualFila.right;
    }

    if (actualFila != cabeceraFila
        && actualFila.columna == columna) {
        actualFila.valor = valor;
        return;
    }

    Nodo cabeceraColumna = columnas[columna];
    Nodo anteriorColumna = cabeceraColumna;
    Nodo actualColumna = cabeceraColumna.down;

    while (actualColumna != cabeceraColumna
        && actualColumna.fila < fila) {
        anteriorColumna = actualColumna;
        actualColumna = actualColumna.down;
    }

    Nodo nuevo = new Nodo(fila, columna, valor);

    nuevo.right = actualFila;
    anteriorFila.right = nuevo;

    nuevo.down = actualColumna;
    anteriorColumna.down = nuevo;

    cantidadNoCeros++;
}
```

Orden pedagógico de la animación

- El primer while muestra la búsqueda horizontal y repite sus líneas por cada salto.
- El segundo while muestra la búsqueda vertical y repite sus líneas por cada salto.
- Las cuatro asignaciones finales se animan una por una; nunca se salta directamente al resultado.
- Si la coordenada ya existe, sólo cambia valor y no aumenta cantidadNoCeros.

6. Código Java educativo - obtener y eliminar

```
public int obtener(int fila, int columna) {
    validarPosicion(fila, columna);

    Nodo cabecera = filas[fila];
    Nodo actual = cabecera.right;

    while (actual != cabecera
        && actual.columna < columna) {
        actual = actual.right;
    }

    if (actual != cabecera
        && actual.columna == columna) {
        return actual.valor;
    }

    return 0;
}

public boolean eliminar(int fila, int columna) {
    validarPosicion(fila, columna);

    Nodo cabeceraFila = filas[fila];
    Nodo anteriorFila = cabeceraFila;
    Nodo objetivo = cabeceraFila.right;

    while (objetivo != cabeceraFila
        && objetivo.columna < columna) {
        anteriorFila = objetivo;
        objetivo = objetivo.right;
    }

    if (objetivo == cabeceraFila
        || objetivo.columna != columna) {
        return false;
    }

    anteriorFila.right = objetivo.right;

    Nodo cabeceraColumna = columnas[columna];
    Nodo anteriorColumna = cabeceraColumna;
    Nodo actualColumna = cabeceraColumna.down;

    while (actualColumna != objetivo) {
        anteriorColumna = actualColumna;
        actualColumna = actualColumna.down;
    }

    anteriorColumna.down = objetivo.down;
    cantidadNoCeros--;
    return true;
}
```

Punto crítico: eliminar exige retirar exactamente el mismo nodo de la fila y de la columna. Si sólo se actualiza right o sólo down, la matriz queda inconsistente. La interfaz debe mostrar ambas desconexiones antes de ocultar el nodo.

7. Código Java educativo - recorridos y validación

```
public void mostrarFila(int fila) {
    if (fila < 0 || fila >= cantidadFilas) {
        throw new IndexOutOfBoundsException("Fila inválida");
    }

    Nodo cabecera = filas[fila];
    Nodo actual = cabecera.right;

    while (actual != cabecera) {
        System.out.println(
            "(" + actual.fila + ", " + actual.columna
            + ") = " + actual.valor
        );
        actual = actual.right;
    }
}

public void mostrarColumna(int columna) {
    if (columna < 0 || columna >= cantidadColumnas) {
        throw new IndexOutOfBoundsException("Columna inválida");
    }

    Nodo cabecera = columnas[columna];
    Nodo actual = cabecera.down;

    while (actual != cabecera) {
        System.out.println(
            "(" + actual.fila + ", " + actual.columna
            + ") = " + actual.valor
        );
        actual = actual.down;
    }
}

private void validarPosicion(int fila, int columna) {
    if (fila < 0 || fila >= cantidadFilas
        || columna < 0 || columna >= cantidadColumnas) {
        throw new IndexOutOfBoundsException(
            "La posición está fuera de la matriz"
        );
    }
}
}
```

Ejemplo básico

```
MatrizDispersa matriz = new MatrizDispersa(4, 5);
matriz.insertar(0, 1, 8);
matriz.insertar(0, 4, 3);
matriz.insertar(2, 0, 6);
matriz.insertar(2, 2, 4);
matriz.insertar(3, 3, 9);

System.out.println(matriz.obtener(2, 2)); // 4
matriz.eliminar(0, 4);
matriz.mostrarFila(2); // 6 y 4
```


8. Plan de integración y pruebas

Para mantener el estándar actual de DSA Lab, la estructura debe incorporarse como un tema completo: datos iniciales, operaciones, código Java, descripción extendida, animación sincronizada, panel de variables y pruebas automáticas.

Piezas de implementación

Área	Responsabilidad
Catálogo	Registrar Matriz dispersa y su categoría, descripción y ejemplo inicial.
Estado	Guardar filas, columnas y lista de celdas {fila, columna, valor}.
Operaciones	Insertar, actualizar, obtener, eliminar y recorrer fila/columna.
Visual	Dibujar cabeceras, nodos, enlaces right/down y retornos circulares.
Código	Exponer todos los métodos usados por cada operación seleccionada.
Animación	Crear frames semánticos y asociarlos con líneas Java reales.
Auditoría	Verificar invariantes después de cada operación.

Suite mínima: 10 pruebas funcionales

- 1. Crear una matriz vacía y comprobar que cada cabecera se apunta a sí misma.
- 2. Insertar la primera celda y verificar right y down.
- 3. Insertar al comienzo, al medio y al final de una fila.
- 4. Insertar al comienzo, al medio y al final de una columna.
- 5. Actualizar una coordenada existente sin aumentar nz.
- 6. Buscar una celda existente y una posición implícitamente igual a cero.
- 7. Eliminar una celda compartida por una fila y una columna con más nodos.
- 8. Eliminar el único nodo de una fila y dejar su cabecera circular.
- 9. Insertar valor cero y confirmar que se elimina la celda.
- 10. Ejecutar al menos 15 inserciones y validar orden, circularidad y ausencia de duplicados.

Resultado del análisis: la propuesta es compatible con la arquitectura actual y es especialmente valiosa para enseñar que una sola referencia incorrecta puede romper dos recorridos. La visualización debe priorizar los enlaces y no solamente la cuadrícula.